

Yes, your current structure is solid — it effectively maps AI tools to real developer workflows (planning/research outside the editor, coding inside it, and collaboration/review at the repo level). It's practical and easy for an audience to relate to.

Recommended Structure: Levels of AI Integration & Autonomy

A **more preferable structure** organizes tools by **depth of integration and capability** rather than just “where” they live. This creates a natural progression:

- It highlights trade-offs (reasoning depth vs. speed vs. seamlessness).
- It scales from simple copy-paste help to near-autonomous assistance.
- It better suits an R/Quarto context (planning complex dashboards/reports → implementing R code + Quarto YAML/chunks → handling multi-file projects like a full interactive app/site).
- It allows easy comparison slides and “when to use which” guidance.

Proposed Presentation Outline (easy to expand into 20–40 min talk):

1. Introduction

- Why AI assistance matters for R/Quarto app development (productivity in data wrangling, viz, interactivity, reproducible reports/dashboards).
- Brief on Quarto specifics (.qmd files, YAML frontmatter, code chunks in R/Python/JS/ Observable, parameters, publishing, Shiny/ interactivity integration).
- Overview of the “Levels” framework + comparison table preview.

2. Level 1: Standalone Conversational AI (your “Standalones”)

3. Level 2: IDE-Integrated Assistants (your “Inside IDE”)

4. Level 3: Agentic & AI-Native Tools (expands capabilities)

5. Level 4: Platform & Collaborative AI (your “Hybrid” + repo-level)

6. Comparison, Best Practices & Recommendations

7. Case Study / Demo Ideas (e.g., building a simple parameterized Quarto sales dashboard)

8. Conclusion & Q&A

This progression feels educational and forward-looking (from lightweight to powerful). You can keep your original labels as subheadings or examples within levels.

Populated Content for Each Level

Level 1: Standalone Conversational AI

Focus: High-level planning, reasoning, learning, and generating complete artifacts. Best when you need deep thought or aren't in the editor yet.

Key Tools: - **Claude** (Anthropic) — Often strongest reasoning and long-context handling. - **Grok** (xAI) — Helpful for real-time or exploratory questions. - **ChatGPT / Gemini** — Widely accessible, good for quick iterations.

R/Quarto Relevance & Use Cases: - Designing app architecture (e.g., Quarto site with multiple pages + shared R data layer + interactivity). - Generating full .qmd templates, YAML configs, or parameterized reports. - Explaining or choosing R packages (tidyverse, ggplot2, plotly, Shiny, DT, etc.). - Troubleshooting complex issues (rendering errors, Quarto extensions, statistical modeling).

Example Prompt (for a sales analytics dashboard app): > “You are an expert in Quarto and R. Design a complete structure for an interactive Quarto-based sales dashboard app. Include: YAML frontmatter with parameters (date range, region), data loading/processing chunks using dplyr and tidyr, visualizations with ggplot2 or plotly, interactivity options (Shiny elements or Observable JS), navigation, and publishing notes. Provide sample code and explain key decisions.”

Strengths: Superior reasoning, excellent for education/explanation, large context windows (Claude excels here), creative solutions.

Limitations: Manual copy-paste; no direct file editing.

Tips: Use Claude Projects/Artifacts for persistent context across sessions. Always verify R code (especially stats/viz logic). Great starting point before moving to IDE tools.

Level 2: IDE-Integrated Assistants

Focus: Real-time help *while coding* — autocomplete, refactoring, contextual chat. Seamless workflow.

Key Tools: - **GitHub Copilot** — Excellent R/Quarto support. - **Supermaven** — Very fast completions with large context window. - Others: Continue.dev (open-source, flexible models), Tabnine, Codeium.

R/Quarto Relevance: - GitHub Copilot has native support in **RStudio/Posit** (since ~2023.09) and works well in VS Code/Positron with the Quarto extension. It understands `.qmd` files, code chunks (switches languages intelligently), YAML, Markdown, and R syntax. - Inline suggestions while writing dplyr pipelines, ggplot2 code, Quarto chunk options (`echo`, `warning`, `cache`, etc.), or site `_quarto.yml`. - Chat sidebar for project-aware questions.

Example Use Cases: - Typing in a `{r}` chunk → Copilot suggests complete `group_by()` + `summarise()` logic based on prior code/comments. - “Add error handling and caching to this Quarto render function.” - Refactoring a shared R utility script used across multiple Quarto pages.

Strengths: Speed and context (open files + project), stays in your editor, good R support via Copilot. Supermaven shines for low-latency completions.

Limitations: Less deep reasoning than standalone tools; suggestions can be generic or occasionally off for niche R packages.

Setup Note: Enable Copilot in RStudio/Posit or VS Code. Works across languages in multi-engine Quarto docs.

Level 3: Agentic & AI-Native Tools

Focus: Tools that understand the *entire codebase* and can plan/edit multiple files, run commands, test, and iterate with less hand-holding.

Key Tools: - **Cursor** (AI-first VS Code fork) — Very popular in 2026. - **Zencoder**, **Windsurf**, **Roo Code** (agentic extensions), advanced Claude features.

R/Quarto Relevance: - Ideal for larger Quarto apps/sites (multiple `.qmd` files + shared R scripts/functions + data pipelines). - “Add a new interactive filtering page, update shared R data utils,

adjust navigation in `_quarto.yml`, and ensure `quarto render` succeeds.” - Can run terminal commands (e.g., `quarto preview`, R tests, package installs) and fix issues autonomously.

Strengths: Handles complexity and consistency across files; reduces context-switching; great for evolving an app over time.

Limitations: Steeper learning curve; higher resource use; still requires oversight (especially for statistical correctness in R).

Best For: Mid-to-large projects where standalone + basic IDE tools feel limiting.

Level 4: Platform & Collaborative AI (Hybrid)

Focus: AI embedded in version control, teams, or platforms — bridges individual work and collaboration.

Key Tools/Features: - GitHub Copilot on GitHub.com (PR reviews, inline suggestions in browser, Workspace-like features). - Posit ecosystem tools (Workbench/Connect AI features where available). - Collaborative environments (e.g., Replit).

R/Quarto Relevance: - AI-assisted code review for R functions or Quarto documents. - Generating PR descriptions or suggesting improvements in context of the full repo. - Team consistency (e.g., enforcing Quarto best practices or R style across contributors).

Strengths: Context of the live repository/team; helps with maintenance and onboarding.

Limitations: Depends on platform adoption; less control than local IDE tools.

Hybrid Value: Complements Levels 1–3 (plan locally → implement → review collaboratively).

Comparison Table (Great Slide)

Level	Reasoning Depth	Context ness	Aware- ness	Speed	R/Quarto port	Sup- port	Best For	Cost/Pri- vacy
1. Standalone	High	Project	descrip- tion	Medium	Excellent (via prompting)	(via	Plan- ning & learn- ing	Varies (API)
2. IDE-Integrated	Medium	File + open project	pro-	High	Strong (esp. Copilot)		Daily coding	Subscrip- tion
3. Agentic	High	Full codebase		Medium-High	Good (with good prompt- ing)	(with prompt-	Com- plex/ multi- file work	Higher

Level	Reasoning Depth	Context ness	Aware-	Speed	R/Quarto port	Sup-	Best For	Cost/Pri- vacy
4.	Plat-Medium form	Repo + team		Medium	Good		Review & col- lab	Platform- dependent

Best Practices & Recommendations (Key Section)

- **Combine levels:** Use Level 1 for architecture → Level 2 for implementation → Level 3 for refactoring → Level 4 for review.
- **Prompt engineering tips** specific to R/Quarto: Be specific about packages/versions, provide sample data structure or error messages, ask for explanations + alternatives, request tests or validation steps.
- **Always verify:** R statistical/visualization code and Quarto rendering. AI can hallucinate package functions or syntax.
- **Privacy:** Important for data apps — consider local models or enterprise plans.
- **R-specific notes:** Copilot and general LLMs handle R well; R has less training data than Python, so explicit context helps. Quarto benefits hugely from AI for YAML and multi-language chunks.
- **Getting started:** Start with GitHub Copilot (easy RStudio integration) + Claude for planning. Experiment with Cursor for agentic feel.
- **Future/Posit angle:** Watch Posit ecosystem AI developments (they're invested in the data science workflow).

This structure is more engaging and insightful than a pure location-based one while still honoring your original categories. It positions you as thoughtful about *how* AI fits into real R/Quarto development rather than just listing tools.

Would you like me to expand any section with more sample prompts, slide bullet points, a full comparison table in markdown, demo script ideas, or visuals (e.g., tool logos or workflow diagrams)? Or help refine it for a specific audience/time length?